

Contents

1 Reliability and Validity

For this chapter we refer to the book Measurement in Medicine.

I invite you to read the introductory chapters 1 and 2 about concepts, theories and models, and types of measurement.

In general, when conducting a measurement of any sort (laboratory measurements, scores from questionnaires, etc.), we want to be reasonably sure

- that we actually **measure what we intend to measure**; (validity; chapter 6 in the book);
- that the measurement does **not change too much** if the underlying **conditions are the same** (reliability; chapter 5 in the book); and
- that we are able to detect a **change** if the underlying conditions change (responsiveness; chapter 7 in the book); and
- that we understand the meaning of a change in the measurement (interpretability; chapter 8 in the book).

In this video, Kai jump starts you on reliability and validity.

1.1 Reliability

You can watch this video to get started.

Imagine, you measure a patient (pick your favorite measurement), for example, the range of motion (ROM) of the shoulder.

- If you are interested in how similar your measurements are in comparison to your colleagues, you are trying to determine the so-called **inter-rater reliability**.
- If you are interested in how similar your measurements are when you measure the same patient twice, you are trying to determine the so-called **intra-rater reliability**.

Assuming there is a true (but unknown) underlying value (of Range of Motion, ROM), it is clear that measurements will not be *exactly* the same. Possible influences (potentially) causing different results are:

- the measurement instrument itself,
- the patient (e.g., mood/motivation),
- the examiner (e.g., mood/focus, influence on patient, interpretation of decision rules),
- the environment (e.g., the room temperature).

Note that the **true score** is defined in our context as the **average** of all measurements if we would measure repeatedly an infinite number of times (p.98). In this sense, one can measure the **true score** (in principle, not in practice) arbitrarily well.

1.1.1 Peter and Mary's ROM measurements

The data can be found here. We randomly select 50 measurements from Peter and Mary in 50 different patients, plot their measurements and annotate the absolutely largest one(s). At first the *not affected shoulder* (nas) and then the *affected shoulder* (as).

```
library(pacman)
p_load(tidyverse, readxl)

# Read file
url <- "https://raw.githubusercontent.com/jdegenfellner/Script_QM2_ZHAW/main/data/chapter%205_assignment1.xlsx"
temp_file <- tempfile(fileext = ".xls")
download.file(url, temp_file, mode = "wb") # mode="wb" is important for binary files
df <- read_excel(temp_file)

head(df)
```

```
## # A tibble: 6 x 5
##   patcode ROMnas.Mary ROMnas.Peter ROMas.Mary ROMas.Peter
##   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1       1         90         92         88         95
## 2       2         82         88         82         90
## 3       3         82         88         57         59
## 4       4         89         89         82         81
## 5       5         80         82         48         40
## 6       6         90         96         99         85
```

```
dim(df)
```

```
## [1] 155  5
```

```
# As in the book, let's randomly select 50 patients.
set.seed(123)
df <- df %>% sample_n(50)
dim(df)
```

```
## [1] 50  5
```

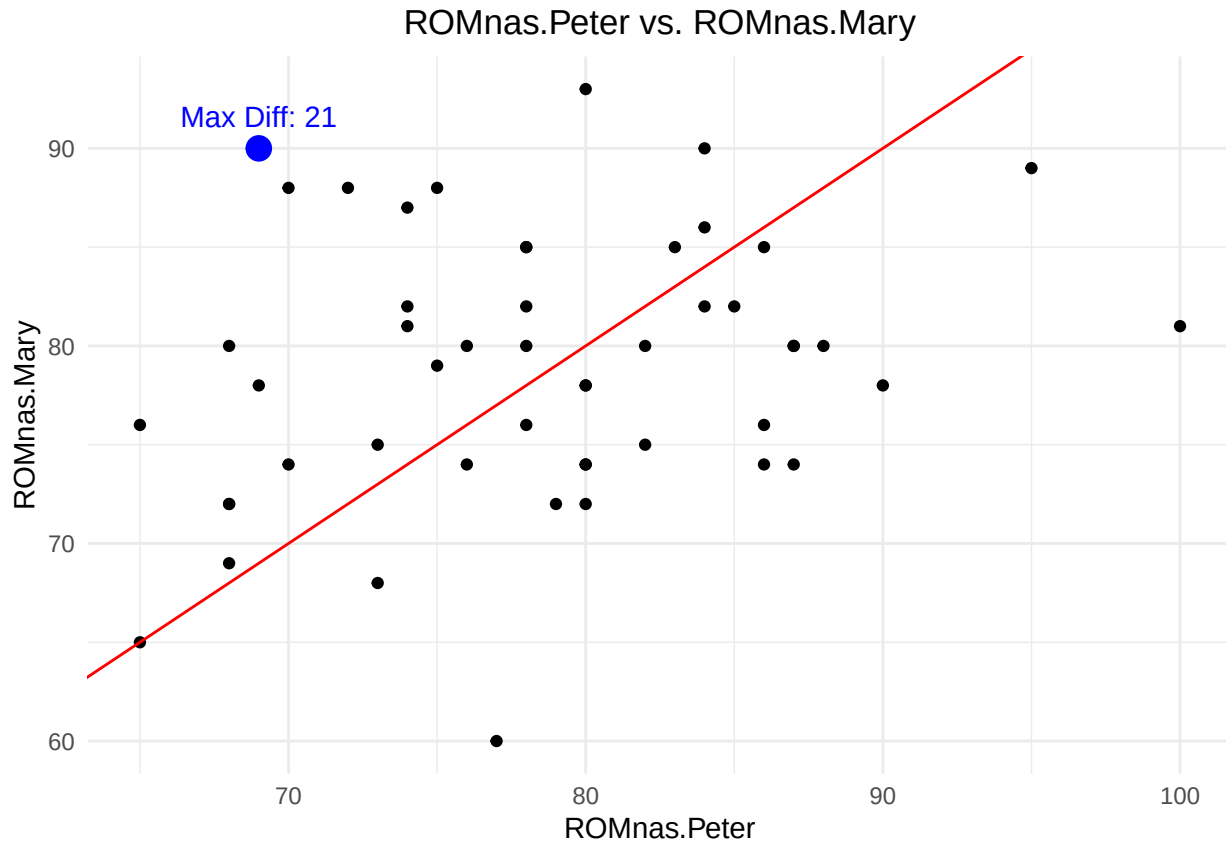
```
# "as" = affected shoulder
# "nas" = not affected shoulder

df <- df %>%
  dplyr::mutate(diff = abs(ROMnas.Peter - ROMnas.Mary)) # Compute absolute difference

max_diff_point <- df %>%
  dplyr::filter(diff == max(diff, na.rm = TRUE)) # Find the row with the max difference

df %>%
  ggplot(aes(x = ROMnas.Peter, y = ROMnas.Mary)) +
  geom_point() +
  geom_point(data = max_diff_point, aes(x = ROMnas.Peter, y = ROMnas.Mary),
            color = "blue", size = 4) + # Highlight max difference point
  geom_abline(intercept = 0, slope = 1, color = "red") +
  theme_minimal() +
  ggtitle("ROMnas.Peter vs. ROMnas.Mary") +
  theme(plot.title = element_text(hjust = 0.5)) +
  annotate("text", x = max_diff_point$ROMnas.Peter,
            y = max_diff_point$ROMnas.Mary,
```

```
label = paste0("Max Diff: ", round(max_diff_point$diff, 2)),
vjust = -1, color = "blue", size = 4)
```



```
# average abs. difference:
mean(df$diff, na.rm = TRUE) # 7.2
```

```
## [1] 7.2
```

```
cor(df$ROMnas.Peter, df$ROMnas.Mary, use = "complete.obs")
```

```
## [1] 0.2403213
```

The red line represents the line of equality ($y = x$). If the measurements are exactly the same, all points would lie on this line ($\rho = 1$). The blue point represents the measurement with the largest absolute difference in measured Range of Motion (ROM) values between Peter and Mary from the randomly chosen 50 people. Note that the maximum difference in all 155 patients is even higher with 35 degrees.

The first simple measure of agreement we could use is the (Pearson) correlation, which measures the **strength and direction of a linear relationship between two variables**. But correlation does not exactly measure what we want. If there was a bias (e.g., Mary systematically measures 5 degrees more than Peter), correlation would not notice this (exercise 1). It actually is too optimistic about the agreement since it only cares about the linearity and not about a potential bias: $r = 0.2403213$ which indicates a weak positive correlation. Higher values of Peter's measurements are associated with higher values of Mary's measurements. But: Knowing Peter's measurement does not help us to *predict* Mary's measurement at such a low correlation (exercise 2). So, on the *not affected shoulder* (nas), agreement is really bad.

What about the affected shoulder (as)?

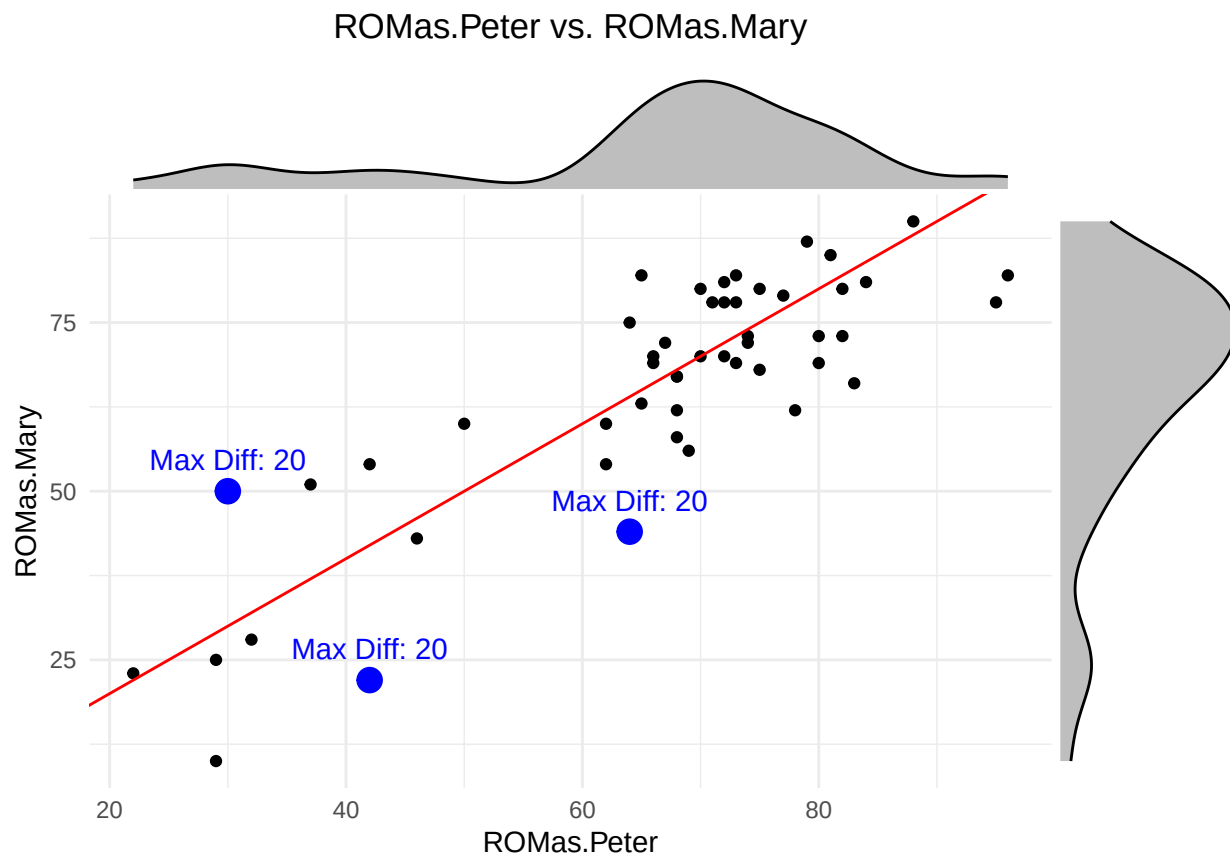
```

library(ggExtra)
df <- df %>%
  mutate(diff = abs(ROMas.Peter - ROMas.Mary)) # Compute absolute difference

max_diff_point <- df %>%
  dplyr::filter(diff == max(diff, na.rm = TRUE)) # Find the row with the max difference

p <- df %>%
  ggplot(aes(x = ROMas.Peter, y = ROMas.Mary)) +
  geom_point() +
  geom_point(data = max_diff_point, aes(x = ROMas.Peter, y = ROMas.Mary),
            color = "blue", size = 4) + # Highlight max difference point
  geom_abline(intercept = 0, slope = 1, color = "red") +
  theme_minimal() +
  ggtitle("ROMas.Peter vs. ROMas.Mary") +
  theme(plot.title = element_text(hjust = 0.5)) +
  annotate("text", x = max_diff_point$ROMas.Peter,
          y = max_diff_point$ROMas.Mary,
          label = paste0("Max Diff: ", round(max_diff_point$diff, 2)),
          vjust = -1, color = "blue", size = 4)
# Add marginal histograms
ggMarginal(p, type = "density", fill = "gray", color = "black")

```



```

# average abs. difference:
mean(df$diff, na.rm = TRUE) # 7.2

```

```
## [1] 7.78
```

```
cor(df$ROMas.Peter, df$ROMas.Mary, use = "complete.obs")
```

```
## [1] 0.8516653
```

```
# mean difference  
mean(df$ROMas.Peter - df$ROMas.Mary, na.rm = TRUE)
```

```
## [1] 1.22
```

In the affected side, the average absolute difference is even larger (7.78) with a maximum absolute difference of 37 degrees (on the whole sample of 155 patients), but the correlation is much higher ($r = 0.8516653$). See Figure 5.2 in the book.

This is an occasion for using the correlation coefficient even though the marginal distributions are not normal: There are much more measurements in the higher values around 80 than below, say, 60. But the correlation coefficient makes sense for descriptive purposes.

In this case, knowing Peter's measurement *does* (at least somewhat) help us to predict Mary's measurement (see below).

1.1.2 Intraclass Correlation Coefficient (ICC)

One way to measure reliability is to use the intraclass correlation coefficient (ICC).

This measure is based on the idea that the observed score Y_i consists of the true score (ROM) (η_i) and a measurement error (for each person). The proportion of the true score variability to the total variability is the ICC.

In the background one thinks of a statistical model from the Classical Test Theory (CTT). There is

- a true underlying score η_i (for each patient i) and
- an error term $\varepsilon_i \sim N(0, \sigma_i)$ which is the difference between the true score and
- the observed score Y_i .

$$Y_i = \eta_i + \varepsilon_i$$

It is assumed that η_i and ε_i are independent: ($\text{Cov}(\eta_i, \varepsilon_i) = 0$). This is a convenient assumption because now we know (see here) that the variance of the observed score Y_i is just the sum of the variance of the true score η_i and the variance of the error term ε_i :

$$\begin{aligned}\text{Var}(Y_i) &= \text{Var}(\eta_i) + \text{Var}(\varepsilon_i) \\ \sigma_{Y_i}^2 &= \sigma_{\eta_i}^2 + \sigma_{\varepsilon_i}^2\end{aligned}$$

We want most of the variability (i.e. how much scores in our relevant population vary) in our observed scores Y_i to be explained by the true but (directly) unobservable scores η_i . The measurement error ε_i should be comparatively small (and of course on average 0). If it is large, we are mostly measuring noise or at least not what we want to measure.

How do we get to the theoretical definition of the ICC?

If you either pull two people with the same true but unobservable score η out of the population or measure the same person twice and the score (η) does not change in between, we can **define reliability as correlation between these two measurements**:

$$Y_1 = \eta + \varepsilon_1$$

$$Y_2 = \eta + \varepsilon_2$$

$$\text{cor}(Y_1, Y_2) = \text{cor}(\eta + \varepsilon_1, \eta + \varepsilon_2) = \frac{\text{Cov}(\eta + \varepsilon_1, \eta + \varepsilon_2)}{\sigma_{Y_1} \sigma_{Y_2}} =$$

If we use

- the properties of the covariance,
- the fact that the true score η and the errors ε_i are independent, and
- the fact that the errors ε_1 and ε_2 are independent, we get:

$$\frac{\text{Cov}(\eta, \eta) + \text{Cov}(\eta, \varepsilon_2) + \text{Cov}(\varepsilon_1, \eta) + \text{Cov}(\varepsilon_1, \varepsilon_2)}{\sigma_{Y_1} \sigma_{Y_2}} = \frac{\sigma_\eta^2 + 0 + 0 + 0}{\sigma_{Y_1} \sigma_{Y_2}}$$

Since η is a random variable (we draw a person randomly from the population), it is well defined to talk about the variance of η (i.e., σ_η^2). I think this aspect may not come across in the book quite so clearly.

Furthermore, it does not matter if I call the measurement Y_1 , Y_2 or more general Y , since they have the same variance and true score:

$$\sigma_Y = \sigma_{Y_1} = \sigma_{Y_2}$$

Hence, it follows that:

$$\text{cor}(Y_1, Y_2) = \frac{\sigma_\eta^2}{\sigma_{Y_1}^2} = \frac{\sigma_\eta^2}{\sigma_Y^2} = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2}$$

This is the **intraclass correlation coefficient (ICC)**. It is (as seen in the formula above) the proportion of the true score variability to the total variability. It ranges from 0 and 1 (think about why!).

A little manipulation to improve understanding:

Let's look again at the term for the ICC above and divide the numerator and the denominator by σ_η^2 , which we can do, since it is a positive number:

$$\frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2} = \frac{1}{1 + \frac{\sigma_\varepsilon^2}{\sigma_\eta^2}}$$

We could call the term $\frac{\sigma_\varepsilon^2}{\sigma_\eta^2}$ the noise-to-signal ratio. The higher this ratio, the lower the ICC. The lower the ratio, the higher the ICC.

- If you increase the noise (measurement error σ_ε^2) for fixed true score variability σ_η^2 , the ICC decreases, because the denominator increases.

- If you increase the true score variability σ_{η}^2 for fixed noise σ_{ε}^2 , the ICC increases, since the denominator decreases.

Btw, we could also divide by σ_{ε}^2 and get the **signal-to-noise ratio**.

At first glance, the following statement seems wrong:

In a very **homogeneous population** (patients have very similar scores/measurements), the **ICC might be very low**. The reason is that the patient variability σ_{η}^2 is low and you probably have some measurement error σ_{ε}^2 . Hence, if you look at the formula, ICC must be low (for a given measurement error).

On the other hand, if you have a very **heterogeneous population** (patients have rather different scores/measurements), the **ICC might be very high**. The reason is that the patient variability σ_{η}^2 is high and you probably have some measurement error σ_{ε}^2 .

What matters is the ratio of the two, as can be seen from the formula above.

Let's try to calculate the ICC for our data using a statistical model. There are a couple of different R packages to do this. We start by using the `irr` package.

```
library(irr)
```

```
## Loading required package: lpSolve
```

```
irr::icc(as.matrix(df[, c("ROMas.Peter", "ROMas.Mary")]),
  model = "oneway", type = "consistency")
```

```
## Single Score Intraclass Correlation
##
## Model: oneway
## Type : consistency
##
## Subjects = 50
## Raters = 2
## ICC(1) = 0.851
##
## F-Test, H0: r0 = 0 ; H1: r0 > 0
## F(49,50) = 12.4 , p = 7.31e-16
##
## 95%-Confidence Interval for ICC Population Values:
## 0.753 < ICC < 0.913
```

```
cor.test(df$ROMas.Peter, df$ROMas.Mary, method = "pearson")
```

```
##
## Pearson's product-moment correlation
##
## data: df$ROMas.Peter and df$ROMas.Mary
## t = 11.259, df = 48, p-value = 4.538e-15
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## 0.7514574 0.9134673
## sample estimates:
## cor
## 0.8516653
```

We get the result: $ICC(1) = 0.851$, which is identical to the correlation coefficient because there is no systematic difference between Peter and Mary.

Since **we are forward looking and modern regression model experts**, we would like to see if we can get (approximately) the same result using the Bayesian framework.

Below is the model structure (see exercise 3).

$$\begin{aligned}
 ROM_i &\sim N(\mu_i, \sigma_\varepsilon) \\
 \mu_i &= \alpha[ID] \\
 \alpha[ID] &\sim \text{Normal}(\mu_\alpha, \sigma_\alpha) \\
 \mu_\alpha &\sim \text{Normal}(66, 20) \\
 \sigma_\alpha &\sim \text{Uniform}(0, 20) \\
 \sigma_\varepsilon &\sim \text{Uniform}(0, 20)
 \end{aligned}$$

Model details:

- ROM_i is the observed ROM -score for patient i . Currently, we have chosen the ROM s to come from a normal distribution. As we have seen above, this might not be adequate, since the marginal distributions look skewed. Every patient has two observations (one each from Mary and Peter). So, for instance $i = 1, 2$ could be patient $ID = 1$.
- μ_i is the expected value of the observed score for patient ID .
- σ_ε is the standard deviation of the measurement error.
- $\alpha[ID]$ is the patient-specific intercept ($= \eta_i$). Since every patient is allowed to have a different intercept, we have a **random intercepts model**. The intercepts are drawn from a normal distribution (hence, *random*)
- μ_α is the mean of the prior for the patient-specific intercepts. This is the overall mean of the scores.
- σ_α is the standard deviation of the patient-specific intercepts. **This is the patient variability!** σ_α controls how much the scores of the patients may vary. In the extreme cases $\sigma_\alpha \rightarrow 0$ and $\sigma_\alpha \rightarrow \infty$, we have *complete-pooling* (all μ_i are identical) and *no-pooling* (all μ_i are different and there is no shared information), respectively. We are using **partial pooling** here.
- The prior distributions express (as always) our prior beliefs about the parameters.

The **ICC** is then calculated as the ratio of the between-patient variance and the total variance:

$$\frac{\sigma_\alpha^2}{\sigma_\alpha^2 + \sigma_\varepsilon^2}$$

We did not even notice it, but this was our first **multilevel regression model**. It is multilevel due to the extra layer of patient-specific intercepts. The **observations** are obviously **clustered within patients**, since observations from the **same patient** are **more similar than observations from different patients**. If one would run a normal linear regression model, one would ignore this clustering and the assumption of independent error terms would be violated.

This time we fire up the **rethinking** package and use the **ulam** function to fit the model. This uses Markov Chain Monte Carlo (MCMC) to sample from the posterior distribution of the parameters.

- The **chains** argument specifies how many chains we want to run. A *chain* is a sequence of points in a space with as many dimensions as there are parameters in the model. It jumps from one point to the next in this parameter space and in doing so, visits the points of the posterior approximately in the correct frequency. Here is an excellent visualization.
- The **cores** argument specifies how many CPU cores we want to use. For larger jobs, one can try to parallelize the chains, which saves some time.


```
library(rethinking)
```

```
## Loading required package: cmdstanr

## This is cmdstanr version 0.8.1.9000

## - CmdStanR documentation and vignettes: mc-stan.org/cmdstanr

## - CmdStan path: /Users/juergen/.cmdstan/cmdstan-2.36.0

## - CmdStan version: 2.36.0

## Loading required package: posterior

## This is posterior version 1.6.1

##
## Attaching package: 'posterior'

## The following objects are masked from 'package:stats':
##
##     mad, sd, var

## The following objects are masked from 'package:base':
##
##     %in%, match

## Loading required package: parallel

## rethinking (Version 2.42)

##
## Attaching package: 'rethinking'

## The following object is masked from 'package:purrr':
##
##     map

## The following object is masked from 'package:stats':
##
##     rstudent
```

```
library(tictoc)
```

```
df_long <- df %>%
  mutate(ID = row_number()) %>%
  dplyr::select(ID, ROMas.Peter, ROMas.Mary) %>%
  pivot_longer(cols = c(ROMas.Peter, ROMas.Mary),
               names_to = "Rater", values_to = "ROM") %>%
```

```

mutate(Rater = factor(Rater))

tic()
m5.1 <- ulam(
  alist(
    # Likelihood
    ROM ~ dnorm(mu, sigma),

    # Patient-specific intercepts (random effects)
    mu <- a[ID],
    a[ID] ~ dnorm(mu_a, sigma_ID), # Hierarchical structure for patients

    # Priors for hyperparameters
    mu_a ~ dnorm(66, 20), # Population-level mean
    sigma_ID ~ dunif(0,20), # Between-patient standard deviation
    sigma ~ dunif(0,20) # Residual standard deviation
  ),
  data = df_long,
  chains = 8, cores = 4
)

```

```
## Running MCMC with 8 chains, at most 4 in parallel, with 1 thread(s) per chain...
```

```

##
## Chain 1 Iteration:   1 / 1000 [  0%] (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 1 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 1 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 1 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 2 Iteration:   1 / 1000 [  0%] (Warmup)
## Chain 2 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 2 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 3 Iteration:   1 / 1000 [  0%] (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 3 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 3 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 3 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%] (Sampling)

```

```

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/pm/jd6')
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 3

## Chain 4 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 4 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 4 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 4 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 4 Iteration: 1000 / 1000 [100%] (Sampling)

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/pm/jd6')
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 4

## Chain 1 finished in 0.1 seconds.
## Chain 3 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 3 finished in 0.2 seconds.
## Chain 4 finished in 0.1 seconds.
## Chain 2 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 2 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 5 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 5 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 5 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 5 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 5 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 5 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 5 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 5 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 5 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 5 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 5 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 5 Iteration: 1000 / 1000 [100%] (Sampling)

```

```

## Chain 5 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 5 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/pm/jd6r')
## Chain 5 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 5 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 5

## Chain 6 Iteration:    1 / 1000 [  0%] (Warmup)
## Chain 6 Iteration:  100 / 1000 [ 10%] (Warmup)
## Chain 6 Iteration:  200 / 1000 [ 20%] (Warmup)
## Chain 6 Iteration:  300 / 1000 [ 30%] (Warmup)
## Chain 6 Iteration:  400 / 1000 [ 40%] (Warmup)
## Chain 6 Iteration:  500 / 1000 [ 50%] (Warmup)
## Chain 6 Iteration:  501 / 1000 [ 50%] (Sampling)
## Chain 6 Iteration:  600 / 1000 [ 60%] (Sampling)
## Chain 6 Iteration:  700 / 1000 [ 70%] (Sampling)

## Chain 6 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 6 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/pm/jd6r')
## Chain 6 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 6 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 6

## Chain 7 Iteration:    1 / 1000 [  0%] (Warmup)
## Chain 7 Iteration:  100 / 1000 [ 10%] (Warmup)
## Chain 7 Iteration:  200 / 1000 [ 20%] (Warmup)
## Chain 7 Iteration:  300 / 1000 [ 30%] (Warmup)
## Chain 7 Iteration:  400 / 1000 [ 40%] (Warmup)

## Chain 7 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 7 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/pm/jd6r')
## Chain 7 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 7 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 7

```

```

## Chain 2 finished in 0.2 seconds.
## Chain 5 finished in 0.1 seconds.
## Chain 6 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 6 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 6 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 7 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 7 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 7 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 7 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 7 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 7 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 7 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 8 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 8 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 8 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 8 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 8 Iteration: 400 / 1000 [ 40%] (Warmup)

## Chain 8 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 8 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/pm/jd6')
## Chain 8 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 8 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 8

## Chain 6 finished in 0.1 seconds.
## Chain 7 finished in 0.1 seconds.
## Chain 8 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 8 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 8 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 8 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 8 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 8 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 8 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 8 finished in 0.1 seconds.
##
## All 8 chains finished successfully.
## Mean chain execution time: 0.1 seconds.
## Total execution time: 0.6 seconds.

toc() # 7s

## 6.923 sec elapsed

precis(m5.1, depth = 2)

##           mean      sd   5.5%   94.5%   rhat ess_bulk
## a[1]  67.801401 4.851443 60.13017 75.60306 1.0024139 6619.991

```

## a[2]	64.190684	4.841765	56.47704	71.96682	1.0033561	6087.696
## a[3]	86.996738	4.736453	79.48393	94.60960	0.9993506	6369.422
## a[4]	76.530318	4.831713	68.84843	84.38513	1.0018190	5157.235
## a[5]	58.776451	4.765509	51.22798	66.39053	1.0018377	5098.476
## a[6]	69.128605	4.965425	61.20783	77.06063	1.0042993	4789.658
## a[7]	69.701335	4.763945	62.04819	77.27248	1.0031944	5659.441
## a[8]	67.333150	4.833474	59.61635	74.95783	1.0013529	5670.597
## a[9]	71.032050	4.905078	63.27131	78.81170	1.0023262	5977.332
## a[10]	80.899118	4.778177	73.26061	88.45832	1.0034929	5550.457
## a[11]	70.487119	4.890308	62.66053	78.19115	1.0024445	7088.019
## a[12]	42.225071	4.891742	34.40218	49.92050	1.0012149	5197.459
## a[13]	86.838805	4.901824	78.94315	94.70806	1.0022252	5415.049
## a[14]	74.588776	4.892254	66.70769	82.28155	1.0061419	6201.765
## a[15]	76.358485	4.765478	68.61011	83.95755	1.0005471	5617.096
## a[16]	34.911261	4.807355	27.39249	42.81919	1.0034562	4996.251
## a[17]	84.690792	4.797903	77.15588	92.52530	1.0019987	5432.365
## a[18]	65.074728	4.886075	57.18416	72.69642	1.0045333	6606.349
## a[19]	63.231107	4.793002	55.48346	70.90928	1.0037856	5676.571
## a[20]	79.697001	4.866240	71.94790	87.56718	1.0035538	5967.776
## a[21]	30.369812	4.899518	22.55073	38.13108	1.0009967	5253.893
## a[22]	62.709968	4.725827	55.22957	70.23577	1.0003572	7404.072
## a[23]	67.427985	4.788840	59.76362	75.10686	1.0029133	5351.814
## a[24]	81.535150	4.741940	73.95096	89.03032	1.0017427	5464.543
## a[25]	55.010214	4.840785	47.42953	62.96822	1.0049586	6085.220
## a[26]	67.434436	4.739068	59.96062	74.95876	1.0021159	6164.406
## a[27]	75.649054	4.913257	67.74207	83.54998	1.0010128	6362.574
## a[28]	81.488379	4.660635	73.92604	88.92539	0.9999582	6327.554
## a[29]	46.253649	4.810893	38.57074	53.79868	1.0049045	5672.224
## a[30]	61.260278	4.673822	53.65320	68.70247	1.0013432	5690.895
## a[31]	23.469053	5.114613	15.36816	31.57722	1.0007722	5354.087
## a[32]	74.082006	4.840605	66.29799	81.77599	1.0009402	5886.113
## a[33]	70.626826	4.823152	62.81915	78.29396	1.0015286	5832.028
## a[34]	76.462800	4.843277	68.56998	84.02210	1.0025194	6883.911
## a[35]	75.572553	4.717920	68.09133	83.12755	1.0058299	5293.603
## a[36]	69.197828	4.764084	61.55910	76.99101	1.0024002	6139.124
## a[37]	49.518817	4.700441	41.98837	57.09118	1.0010456	4710.058
## a[38]	73.710164	4.932679	65.69411	81.61300	1.0028124	6378.396
## a[39]	72.772866	4.783717	65.00724	80.30890	1.0055246	6250.379
## a[40]	45.785394	4.846582	38.04010	53.37593	1.0015821	6324.488
## a[41]	73.738559	4.887535	65.93367	81.72445	1.0012916	6199.265
## a[42]	26.216631	5.083953	18.26280	34.61207	1.0030459	5052.563
## a[43]	33.121094	4.850825	25.59334	40.87299	1.0010879	5411.410
## a[44]	74.198091	4.682154	66.69696	81.59399	1.0070692	6485.594
## a[45]	76.973015	4.874692	69.15629	84.69509	1.0010603	5720.713
## a[46]	73.689479	4.849046	65.80651	81.37046	1.0026245	4437.984
## a[47]	55.951497	4.749996	48.36908	63.62176	1.0014202	5516.828
## a[48]	69.715662	4.820367	62.00446	77.43062	1.0025555	5772.378
## a[49]	72.362550	4.812730	64.57572	80.00205	0.9998971	5552.498
## a[50]	72.711909	4.930061	64.73394	80.64126	1.0012210	5531.604
## mu_a	65.586927	2.418753	61.82518	69.51800	1.0019391	5120.128
## sigma_ID	16.568114	1.591735	13.99826	19.14733	1.0027477	3039.192
## sigma	7.077867	0.730829	6.00359	8.31929	1.0035903	1612.349

```
post <- extract.samples(m5.1)
var_patients <- mean(post$sigma_ID^2) # Between-patient variance
var_residual <- mean(post$sigma^2)   # Residual variance
var_patients / (var_patients + var_residual) # ICC
```

```
## [1] 0.8454822
```

```
# 0.846
# not too bad; very close to the result from the irr package
```

In the output from `precis(m5.1, depth = 2)` above we see

- all 50 intercept estimates for each patient: `a[ID]`
- `mu_ais` the overall intercept.
- `sigma_ID` is the **patient variability**.
- `sigma` is the **residual variability**.

We have just squared the sigmas to get the variances.

Remember: In the background of all ICCs, there is always a statistical model to predict the outcome. Depending on the predictors, we get different models and probably different ICCs.

Let's jump back into the Frequentist world and see if we can get the same result.

We can also estimate a **random intercept model** with the `lme4` package using the command `lmer` in the Frequentist framework. No priors:

```
library(lme4)
```

```
## Loading required package: Matrix
```

```
##
```

```
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
```

```
##
```

```
##      expand, pack, unpack
```

```
m5.2 <- lmer(ROM ~ (1|ID), data = df_long)
summary(m5.2)
```

```
## Linear mixed model fit by REML ['lmerMod']
```

```
## Formula: ROM ~ (1 | ID)
```

```
## Data: df_long
```

```
##
```

```
## REML criterion at convergence: 791
```

```
##
```

```
## Scaled residuals:
```

```
##      Min      1Q   Median      3Q      Max
## -1.91875 -0.44821  0.00964  0.51325  1.47941
```

```
##
```

```
## Random effects:
## Groups   Name          Variance Std.Dev.
## ID       (Intercept) 270.99   16.462
## Residual                47.35    6.881
## Number of obs: 100, groups: ID, 50
##
## Fixed effects:
##              Estimate Std. Error t value
## (Intercept)   65.590      2.428   27.02
```

```
print(VarCorr(m5.2), comp = "Variance")
```

```
## Groups   Name          Variance
## ID       (Intercept) 270.99
## Residual                47.35
```

```
# ICC =
270.99 / (270.99 + 47.35) #
```

```
## [1] 0.8512597
```

```
# 0.8512597
# -> exactly the same result as the irr package
```

The expression `Formula: ROM ~ (1 | ID)` specifies that we want to fit a model with a random intercept. This means that every patient (ID) gets its own intercept which is drawn from a normal distribution.

So far, we have only looked at the general *ICC* (*ICC1* in the **psych** output) (see also page 106 in the book).

There, we have not yet explicitly considered a bias (=systematic difference between the raters) that the raters could have. In the book, they introduce a bias of 5 degrees (Mary measures 5 degrees more than Peter on average).

The **model für ICC 2 and 3** in the **psych** output explicitly considers this (systematic) difference that could occur between the raters. This results in an extra term in the denominator of the *ICC*, an additional variance component.

From the **same statistical model** (!) we take the variance components to calculate:

$$ICC_{agreement} = \frac{\sigma_{\alpha}^2}{\sigma_{\alpha}^2 + 2_{\text{rater}} + \sigma_{\varepsilon}^2}$$

where σ_{rater}^2 is the variance due to systematic rater differences.

$$ICC_{consistency} = \frac{\sigma_{\alpha}^2}{\sigma_{\alpha}^2 + \sigma_{\varepsilon}^2}$$

We will now introduce the 5 degree bias and use our Bayesian framework to estimate the *ICC*. By introducing a bias, we should see a lower *ICC*. Note, that the prediction quality of Mary's scores given Peter's scores should not change, since we would only shift Mary's scores down by 5 degrees, which would not disturb the linear regression model. We can always shift the points to where we want them to be. We do that for instance when we scale or standardize the data.

Admitted, the Bayesian version in this case takes longer and is more complex. The advantage is still that it's fully probabilistic and one could work with detailed prior information, especially for smaller sample sizes.

Anyhow, let's try to give the model equations considering the introduced bias. This is the model **for both** $ICC_{agreement}$ and $ICC_{consistency}$!

$$\begin{aligned}
 ROM_i &\sim N(\mu_i, \sigma_\varepsilon) \\
 \mu_i &= \alpha[ID] + \beta[Rater] \\
 \alpha[ID] &\sim \text{Normal}(\mu_\alpha, \sigma_\alpha) \\
 \beta[Rater] &\sim \text{Normal}(0, \sigma_\beta) \\
 \mu_\alpha &\sim \text{Normal}(66, 20) \\
 \sigma_\alpha &\sim \text{Exp}(0.5) \\
 \sigma_\beta &\sim \text{Exp}(1) \\
 \sigma_\varepsilon &\sim \text{Exp}(1)
 \end{aligned}$$

As you can see, μ_i now consists of the patient-specific intercept $\alpha[ID]$ (everyone of the 50 patients gets one) and the rater-specific effect $\beta[Rater]$ (Mary and Peter get one). So, in total, we have **three sources of variability**:

- the patient variability σ_α ,
- the rater variability σ_β ,
- and the residual variability σ_ε .

Note, that if Peter measures each of the 50 patients twice, the systematic difference between Peter's measurements would be zero. Of course, one could be creative and think of a learning effect or something.

Draw the model structure as exercise 5.

```
library(rethinking)
library(conflicted)
conflicts_prefer(posterior::sd)
```

```
## [conflicted] Will prefer posterior::sd over any other package.
```

```
df_long_bias <- df_long %>%
  mutate(ROM = ROM + ifelse(Rater == "ROMas.Mary", 5, 0))
head(df_long_bias)
```

```
## # A tibble: 6 x 3
##   ID Rater      ROM
##   <int> <fct>   <dbl>
## 1     1 ROMas.Peter 66
## 2     1 ROMas.Mary 75
## 3     2 ROMas.Peter 65
## 4     2 ROMas.Mary 68
## 5     3 ROMas.Peter 96
## 6     3 ROMas.Mary 87
```

```
set.seed(123)
m5.2 <- ulam(
  alist(
    # Likelihood
    ROM ~ dnorm(mu, sigma_eps),

    # Model for mean ROM with patient and rater effects
```

```

mu <- alpha[ID] + beta[Rater],

# Patient-specific random effects
alpha[ID] ~ dnorm(mu_alpha, sigma_alpha),

# Rater effect (Peter/Mary)
beta[Rater] ~ dnorm(0, sigma_beta),

# Priors for hyperparameters
mu_alpha ~ dnorm(66, 10), # Population mean ROM
sigma_alpha ~ dexp(0.5), # Between-patient SD (less aggressive shrinkage)
sigma_beta ~ dexp(1), # Rater SD (better regularization)
sigma_eps ~ dexp(1) # Residual SD (prevents over-shrinkage)
),
data = df_long_bias,
chains = 8, cores = 4
)

## Running MCMC with 8 chains, at most 4 in parallel, with 1 thread(s) per chain...
##
## Chain 1 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 2 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 3 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%] (Warmup)

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/pm/jd6r')
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 3

## Chain 4 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 1 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 1 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 2 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 2 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 2 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 2 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%] (Sampling)

```

```

## Chain 3 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 3 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 3 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 3 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 4 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 4 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 4 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 2 finished in 0.2 seconds.
## Chain 3 finished in 0.2 seconds.
## Chain 1 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 4 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 5 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 5 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 5 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 6 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 1 finished in 0.3 seconds.
## Chain 4 finished in 0.3 seconds.
## Chain 5 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 5 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 5 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 5 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 5 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 5 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 5 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 5 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 5 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 6 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 6 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 6 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 6 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 6 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 6 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 6 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 6 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 6 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 6 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 6 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 7 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 7 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 8 Iteration: 1 / 1000 [ 0%] (Warmup)

```

```
## Chain 8 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 8 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/pm/jd6r')
## Chain 8 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 8 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 8
```

```
## Chain 5 finished in 0.2 seconds.
## Chain 6 finished in 0.2 seconds.
## Chain 7 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 7 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 7 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 7 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 7 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 7 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 7 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 7 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 7 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 8 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 8 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 8 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 8 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 8 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 8 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 8 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 8 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 7 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 8 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 8 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 8 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 7 finished in 0.2 seconds.
## Chain 8 finished in 0.3 seconds.
##
## All 8 chains finished successfully.
## Mean chain execution time: 0.2 seconds.
## Total execution time: 0.8 seconds.
```

```
## Warning: 32 of 4000 (1.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.
```

```
precis(m5.2, depth = 2)
```

	mean	sd	5.5%	94.5%	rhat	ess_bulk
## alpha[1]	70.098372	4.7042991	62.6322845	77.6520050	1.001530	4043.560
## alpha[2]	66.569489	4.7693775	59.0935945	74.1363355	1.001763	3908.733
## alpha[3]	89.457889	4.6266261	82.0659185	96.5845055	1.001296	3557.326
## alpha[4]	78.863009	4.6795836	71.4526810	86.5071480	1.001155	3872.797
## alpha[5]	61.160273	4.8743939	53.3981645	68.9984655	1.005173	4221.798
## alpha[6]	71.570030	4.7791323	63.7869920	79.2358775	1.001868	4220.747
## alpha[7]	72.059336	4.7721724	64.4189790	79.7012990	1.000718	3938.921
## alpha[8]	69.792439	4.7315623	62.3561175	77.4358765	1.004061	4836.922
## alpha[9]	73.320955	4.6129611	65.7833975	80.7233240	1.002518	4349.003
## alpha[10]	83.397924	4.7182611	75.7780945	91.0030155	1.003485	4263.796
## alpha[11]	72.852797	4.5797786	65.5314130	80.2417660	1.001249	4967.112

```

## alpha[12] 44.784018 4.9074767 37.0184000 52.6481990 1.001981 5105.934
## alpha[13] 89.297771 4.7002217 81.6948340 96.7268745 1.002795 4083.770
## alpha[14] 77.059998 4.8425231 69.3173470 84.8155935 1.002627 3369.333
## alpha[15] 78.775251 4.6338086 71.4220010 86.1287545 1.002519 4491.709
## alpha[16] 37.435443 4.7659227 29.9392865 45.1209270 1.001814 3856.455
## alpha[17] 86.933759 4.7616634 79.3303120 94.4104210 1.002306 3681.805
## alpha[18] 67.437836 4.7736061 59.9265010 75.1330260 1.004932 4445.400
## alpha[19] 65.725657 4.7193609 58.3514780 73.2018035 1.000595 3350.151
## alpha[20] 82.091286 4.8009036 74.3103950 89.6811220 1.004743 4093.597
## alpha[21] 32.817934 4.7967889 25.1070865 40.5885315 1.000936 3666.192
## alpha[22] 65.211938 4.7906230 57.7547720 72.7764005 1.000964 4475.820
## alpha[23] 69.718651 4.7591161 62.0347805 77.2424650 1.003816 4337.102
## alpha[24] 83.907862 4.6610319 76.3731975 91.1301860 1.002046 3808.518
## alpha[25] 57.451790 4.7134912 49.8447075 64.9600665 1.001046 3569.522
## alpha[26] 69.738023 4.7361684 62.1954645 77.3361665 1.001038 4163.223
## alpha[27] 77.935815 4.5854233 70.6915745 85.0976475 1.002237 3059.448
## alpha[28] 83.948403 4.7068664 76.4390250 91.4390530 1.004254 3594.525
## alpha[29] 48.762531 4.6822810 41.3137875 56.2599180 1.003356 3548.471
## alpha[30] 63.821278 4.6746985 56.3676810 71.4452815 1.001947 3289.059
## alpha[31] 26.016448 4.8900457 18.4122570 33.9521325 1.002461 3510.190
## alpha[32] 76.507025 4.7626648 68.8128075 84.1518680 1.001263 3860.511
## alpha[33] 72.904999 4.8802530 65.1628015 80.7713440 1.004172 4655.722
## alpha[34] 78.781812 4.7523672 71.0831555 86.5075635 1.000770 4146.453
## alpha[35] 77.862658 4.7946130 70.3506130 85.5765545 1.001998 4465.411
## alpha[36] 71.573811 4.8000164 63.9453735 79.2936730 1.001894 4075.348
## alpha[37] 51.967007 4.6390314 44.5768340 59.4492165 1.003622 3052.391
## alpha[38] 76.216932 4.6867388 68.6011415 83.5097200 1.000684 4009.291
## alpha[39] 75.186970 4.7760475 67.6954900 82.7756935 1.000853 4225.980
## alpha[40] 48.313854 4.7909876 40.7513665 55.8724120 1.006659 3989.731
## alpha[41] 76.106722 4.9004359 68.2118730 83.8790650 1.006167 3959.068
## alpha[42] 28.860624 4.7804416 21.0606160 36.4982260 1.000772 3845.006
## alpha[43] 35.551375 4.7650211 28.0875875 43.2557090 1.003077 3645.726
## alpha[44] 76.591764 4.7437801 69.2447800 84.3623410 1.002210 3825.927
## alpha[45] 79.246903 4.7335075 71.5678685 86.8304860 1.002345 3067.135
## alpha[46] 76.098305 4.6831257 68.8458735 83.4560000 1.002134 3819.541
## alpha[47] 58.340678 4.5995812 51.1278315 65.6481140 1.004645 3747.535
## alpha[48] 71.968853 4.6893025 64.4579350 79.4666705 1.002250 3229.449
## alpha[49] 74.723807 4.6761841 67.3960390 82.2802220 1.005403 4461.720
## alpha[50] 75.176016 4.7992669 67.7644245 82.8261675 1.003107 4355.544
## beta[1] 1.299615 1.5127533 -0.6688352 3.9638292 1.010280 1068.607
## beta[2] -1.167443 1.4935396 -3.7346674 0.8214701 1.007193 1080.743
## mu_alpha 67.876858 2.5564075 63.8479340 71.9241770 1.005497 1898.765
## sigma_alpha 15.393556 1.5603445 13.0410405 17.9750610 1.000238 4190.714
## sigma_beta 1.679613 1.0172190 0.4076696 3.4572703 1.004039 1704.538
## sigma_eps 6.733778 0.6484083 5.7829221 7.8311868 1.003769 2009.472

```

```
precis(m5.2)
```

```
## 52 vector or matrix parameters hidden. Use depth=2 to show them.
```

```

##          mean      sd      5.5%      94.5%      rhat ess_bulk
## mu_alpha 67.876858 2.5564075 63.8479340 71.924177 1.005497 1898.765
## sigma_alpha 15.393556 1.5603445 13.0410405 17.975061 1.000238 4190.714

```

```
## sigma_beta    1.679613 1.0172190 0.4076696 3.457270 1.004039 1704.538
## sigma_eps     6.733778 0.6484083 5.7829221 7.831187 1.003769 2009.472
```

```
# check systematic difference for rater in posterior
post <- extract.samples(m5.2)
mean(post$beta[,1] - post$beta[,2])
```

```
## [1] 2.467058
```

```
# ICC agreement:
post <- extract.samples(m5.2)
(var_patients <- mean(post$sigma_alpha^2)) # Between-patient variance
```

```
## [1] 239.3956
```

```
(var_raters <- mean(post$sigma_beta^2)) # Rater variance
```

```
## [1] 3.855577
```

```
(var_residual <- mean(post$sigma_eps^2)) # Residual variance
```

```
## [1] 45.7641
```

```
# ICC_agreement =
var_patients / (var_patients + var_raters + var_residual)
```

```
## [1] 0.8283147
```

```
# 0.8033613 (sigma_alpha ~ dexp(1))
# 0.83 (sigma_alpha ~ dexp(0.5))

# ICC (Single_fixed_raters) = ICC3 in psych output =
var_patients / (var_patients + var_residual)
```

```
## [1] 0.8395142
```

```
# 0.8415256
```

It should be noted that this ICC is very sensitive to the choice of the prior. If you choose too aggressive priors for the standard deviations $\sigma_\alpha, \sigma_\beta, \sigma_\epsilon$, you will get a too low ICC.

I have played around a little with the parameters in the exponential priors to get the desired result which compares nicely to the two alternative methods below: using the **psych** package and with the **lmer** package. Both use a Frequentist random intercept model in the background. Using a package like **psych** gives a rather convenient interface to elicit the ICC. It can never hurt to peek behind the curtain a little bit though.

psych package (Frequentist):

```
library(psych)
library(conflicted)
# needs wide format
#conflicts_prefer(dplyr::select)
df_wide <- df_long_bias %>%
  pivot_wider(names_from = Rater, values_from = ROM)
df_wide_values <- df_wide %>% dplyr::select(-ID)
psych::ICC(df_wide_values) # ICC1 = 0.83
```

```
## Call: psych::ICC(x = df_wide_values)
##
## Intraclass correlation coefficients
##
```

	type	ICC	F	df1	df2	p	lower bound	upper bound
## Single_raters_absolute	ICC1	0.83	11	49	50	1.1e-14	0.72	0.90
## Single_random_raters	ICC2	0.83	12	49	49	1.4e-15	0.71	0.91
## Single_fixed_raters	ICC3	0.85	12	49	49	1.4e-15	0.75	0.91
## Average_raters_absolute	ICC1k	0.91	11	49	50	1.1e-14	0.84	0.95
## Average_random_raters	ICC2k	0.91	12	49	49	1.4e-15	0.83	0.95
## Average_fixed_raters	ICC3k	0.92	12	49	49	1.4e-15	0.86	0.95

```
##
## Number of subjects = 50      Number of Judges = 2
## See the help file for a discussion of the other 4 McGraw and Wong estimates,
```

lmer package:

```
# _lmer-----
m5.3 <- lmer(ROM ~ (1 | ID) + (1 | Rater), data = df_long_bias)
summary(m5.3)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: ROM ~ (1 | ID) + (1 | Rater)
## Data: df_long_bias
##
## REML criterion at convergence: 793.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.87448 -0.46270  0.00272  0.57820  1.45008
##
## Random effects:
## Groups   Name                Variance Std.Dev.
## ID       (Intercept) 270.882   16.458
## Rater     (Intercept)   6.193    2.489
## Residual                    47.557    6.896
## Number of obs: 100, groups: ID, 50; Rater, 2
##
## Fixed effects:
##              Estimate Std. Error t value
## (Intercept)   68.090      2.998   22.71
```

```
print(VarCorr(m5.3), comp = "Variance")
```

```
## Groups   Name                Variance
## ID       (Intercept) 270.882
## Rater    (Intercept)   6.193
## Residual                        47.557
```

```
# Groups   Name                Variance
# ID       (Intercept) 270.882
# Rater    (Intercept)   6.193
# Residual                        47.557
```

```
# ICC (Single_random_raters) = ICC2 in psych output
270.882 / (270.882 + 6.193 + 47.557) #
```

```
## [1] 0.8344279
```

```
# 0.8344279
```

```
# ICC (Single_fixed_raters) = ICC3 in psych output
270.882 / (270.882 + 47.557) #
```

```
## [1] 0.8506559
```

```
# 0.85
```

There are basically **two different random effects**:

- **Nested random effects:** The patients (respectively their *ROMs*) would cluster within the raters. This would mean, a patient would only be measured by Peter twice or by Mary twice. In that case, the *ROMs* would cluster within the patients (because of the repeated measurement) and all patients measured by (for instance) Mary would cluster together (and be potentially more similar). In our case at hand all patients are measured by Peter and Mary, so we do not have nested random effects. In `lmer` nested effects would be specified as `ROM ~ (1|Rater|ID)`.
- **Crossed random effects:** Not nested. In R this would be specified as `ROM ~ (1|ID) + (1|Rater)`.

1.1.3 Explanation of ICCs in the psych output

If you want to know all the details, refer to Shrout and Fleiss (1979). The help-function `?psych::ICC` contains a relatively good and much shorter explanation. The variance formulae given in the help-file are probably somewhat confusing. We try to stick to the notation of the book.

Let's talk about the first three **ICCs in the psych output**:

- **Single_raters_absolute ICC1:** According to the help file: "Each target [i.e. patient] is rated by a different judge [i.e. rater] and the judges are selected at random." So, variability due to raters is implied and cannot be disentangled. This is formally not our situation, since we have only two raters and 50 patients. But for this case, we do not care who measures, since we do not model it, hence, we

cannot know if there are systematic differences between the raters. There might as well be 50 raters doing their thing, or just 2 as in our case. This is the ICC we have calculated above; the overall ICC in the book and it is based on the following model:

$$Y_{ij} = \eta_i + \varepsilon_i$$

where $i \in 1, \dots, 50$ is the *patient* and $j \in 1, 2$ is the *measurement* ($= 50 * 2 = 100$ rows in long format). Note that we do not mention a rater here, since we do not care who took the measurement. It is not part of the model. The ICC is then calculated as:

$$ICC = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2}$$

whereas we could get the variance components from either the posterior in the Bayesian setting or from the `lmer` output in the Frequentist setting.

- **Single_random_raters ICC2:** ICC2 ($= ICC_{agreement}$ in the book) and ICC3 ($= ICC_{consistency}$ in the book) are based on the **same (!)** statistical model. The only difference is that ICC2 assumes that the (in our case) 2 raters are randomly selected from a larger pool of raters, hence, the rater variability must be explicitly considered and yields a potentially smaller value for the ICC. Compared to ICC1, we have repeated measurements from the same raters in 50 patients. That's why we can model their bias. One observation would not be enough. The help file says: "A random sample of k judges rate each target. The measure is one of absolute agreement in the ratings." A random sample of k (2 in our case) judges means that we cannot rule out the variability due to raters (you get a variety of them and their biases are different).

$$Y_{ij} = \eta_i + \beta_j + \varepsilon_i$$

where $i \in 1, \dots, 50$ is the *patient*, $j \in 1, 2$ is the **rater** (doing one measurement in each patient). The ICC is then calculated as:

$$ICC_{agreement} = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \mathbf{2_{rater}} + \sigma_\varepsilon^2}$$

- **Single_fixed_raters ICC3:** ICC3 ($= ICC_{consistency}$ in the book) is based on the **same model as ICC2**, but assumes that the raters are fixed. This means that **the raters are the same for all patients** in the future study. So, we have considered the rater variability in the model (which was possible due to the repeated measurements from the same raters in 50 patients), but do not care since Mary and Peter will be the people doing the future measurements, not other therapists. If you fix a random variable (*raters* in this case), variance is zero. The help file says: "A fixed set of k judges rate each target. There is no generalization to a larger population of judges." The ICC is then calculated as:

$$ICC_{consistency} = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2}$$

Important: ICC1 and ICC3 are **not** identical, since ICC1 does not consider the rater variability in the model. They are based on *different* statistical models. ICC2 and ICC3 are based on the *same* model.

If there is no systematic difference between raters, all 3 ICCs and the Pearson correlation (r) are the same (see Figure 5.3 in the book).

$ICC_{consistency}$ vs. $ICC_{agreement}$: The latter is used, when we need Peter and Mary to concur in their measurements. Patients coming to Peters practice will get the same (or very similar) “diagnosis” (ROM-value) from Mary. When there is systematic difference (line is shifted downwards in Figure 5.3), this cannot be guaranteed. If we only need Peter and Mary to *rank* the patients in the same order, we can use $ICC_{consistency}$.

- **Average_raters_absolute ICC1k:** According to the help file “ICC1k, ICC2k, ICC3k reflect the means of k raters.” In general, if you take the mean, you get an estimate with lower variance (central limit theorem). In the book on page 109, we find the derivation of the ICC for the average of k raters. We do not need a new statistical model, although we take the sum (or equivalently, the mean) of the measurements of Peter and Mary, which seems a bit unintuitive at first.

If we go back to the definition of the ICC above, it follows that:

$$Y_1 + Y_2 = 2\eta + \varepsilon_1 + \varepsilon_2$$

for the same person with true η . Since the errors ε_i are independent and the true score η is also independent from the errors, it follows that:

$$\begin{aligned}\text{Var}(Y_1 + Y_2) &= \text{Var}(2\eta + \varepsilon_1 + \varepsilon_2) = \\ \text{Var}(2\eta) + \text{Var}(\varepsilon_1 + \varepsilon_2) &= \\ 4\text{Var}(\eta) + 2\text{Var}(\varepsilon) &= \end{aligned}$$

If we divide by 4, we get:

$$\frac{1}{4}\text{Var}(Y_1 + Y_2) = \text{Var}\left(\frac{Y_1 + Y_2}{2}\right) = \text{Var}(\eta) + \frac{\text{Var}(\varepsilon)}{2} = \sigma_\eta^2 + \frac{\sigma_\varepsilon^2}{2}$$

Hence, the ICC for the average of 2 raters is ($k = 2$):

$$ICC1k = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \frac{\sigma_\varepsilon^2}{2}}$$

Again, the model does not care about the raters, it just knows that there are two measurements. A potential bias between raters is not considered.

It is also interesting to note that we do not fit a new model to get the ICC1k. We could not compare the first average (of 2 measurements) to the second average (of 2 measurements), since we only have 2 measurements per patient.

```
m <- lmer(ROM ~ (1|ID), data = df_long_bias)
summary(m)

## Linear mixed model fit by REML ['lmerMod']
## Formula: ROM ~ (1 | ID)
## Data: df_long_bias
##
## REML criterion at convergence: 797.3
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.02334 -0.52483 -0.03788  0.57830  1.59879
##
## Random effects:
## Groups   Name      Variance Std.Dev.
```

```
## ID      (Intercept) 267.79   16.364
## Residual           53.75    7.331
## Number of obs: 100, groups: ID, 50
##
## Fixed effects:
##           Estimate Std. Error t value
## (Intercept)  68.090      2.428   28.05
```

```
print(VarCorr(m), comp = "Variance")
```

```
## Groups  Name      Variance
## ID      (Intercept) 267.79
## Residual           53.75
```

```
# ICC1k =
267.79 / (267.79 + 53.75/2)
```

```
## [1] 0.9087947
```

This is identical to the ICC1k in the psych output.

ICC1k would approach 1 if k goes to infinity. **This makes sense**, since the *true* measurement is *defined* as the average of infinite measurements (of either infinitely many different raters or infinitely many measurements from the same rater). Also it makes intuitive sense, that reliability increases or equivalently the unexplained variance decreases since we draw repeatedly from the same distribution (η and an error term ε) and the central limit theorem guarantees that we can estimate η as precisely as we want (variance of the mean goes to zero) if we only draw often enough.

- **Average_raters_random ICC2/3k:** ICC2k and ICC3k are based on the same model as ICC2 and ICC3, respectively. We just need to divide the respective residual variance by k (in our case 2) to get the ICC for the average of k raters.

```
m5.3 <- lmer(ROM ~ (1 | ID) + (1 | Rater), data = df_long_bias)
summary(m5.3)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: ROM ~ (1 | ID) + (1 | Rater)
## Data: df_long_bias
##
## REML criterion at convergence: 793.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.87448 -0.46270  0.00272  0.57820  1.45008
##
## Random effects:
## Groups  Name      Variance Std.Dev.
## ID      (Intercept) 270.882   16.458
## Rater   (Intercept)  6.193    2.489
## Residual           47.557    6.896
## Number of obs: 100, groups: ID, 50; Rater, 2
##
```

```
## Fixed effects:
##           Estimate Std. Error t value
## (Intercept)  68.090      2.998   22.71
```

```
print(VarCorr(m5.3), comp = "Variance")
```

```
## Groups   Name      Variance
## ID       (Intercept) 270.882
## Rater    (Intercept)  6.193
## Residual                47.557
```

```
# Groups   Name      Variance
# ID       (Intercept) 270.882
# Rater    (Intercept)  6.193
# Residual                47.557

# ICC2k =
270.882 / (270.882 + (6.193 + 47.557) / 2)
```

```
## [1] 0.9097418
```

```
# ICC3k =
270.882 / (270.882 + 47.557 / 2)
```

```
## [1] 0.919302
```

These values are identical to the ICC2k and ICC3k in the `psych` output.

1.1.4 Summary Peter and Mary, with and without bias (ICC1-3)

Below, we summarize the results for the ICCs (calculated with `psych`) for the unbiased and biased case (Mary measures on average 5 degrees more than peter).

```
library(pacman)
p_load(conflicted, tidyverse, flextable)

# Ensure select() from dplyr is used
#conflicted::conflicts_prefer("select", "dplyr")

# Unbiased ICC Calculation
df_wide_unbiased <- df_long %>%
  pivot_wider(names_from = Rater, values_from = ROM)
df_wide_values_unbiased <- df_wide_unbiased %>% dplyr::select(-ID)
icc_results_unbiased <- psych::ICC(df_wide_values_unbiased)
```

```
## boundary (singular) fit: see help('isSingular')
```